

aws

19

Important

AWS

CLI

commands



Ömer Berat Sezer

PhD | Software Engineer | DevOps |
Solution Architect

linkedin.com/in/omerberatsezer/

github.com/omerbsezer

Normally, Infrastructure as Code (IaC) is used to provision and configure the AWS services like Terraform or CloudFormation.

In this note, popular and some of the AWS CLI (command line interface) commands that are used for debugging, listing, monitoring, solving issue, and testing, are listed.

AWS CLI Command Reference:

<https://docs.aws.amazon.com/cli/latest/reference/>



AWS Configure

```
user@aws:$ aws configure                                # interactive setup
AWS Access Key ID [None]: AKIAEXAMPLE
Secret Access Key [None]: wJalrXUtnFEMI
Default region name [None]: eu-central-1
Default output format [None]: json
user@aws:$ aws configure --profile myprofile
# to set up multiple AWS configurations within the same environment
user@aws:$ aws configure list-profiles                  # list all profiles
user@aws:$ aws configure list --profile myprofile
# view profile configuration
```



EC2

```
user@aws:$ aws ec2 describe-instances      # list all EC2 VM instances
```

```
user@aws:$ aws ec2 describe-instances --debug
```

```
# add --debug to enable debug mode
```

```
user@aws:$ aws ec2 start-instances --instance-ids i-0123456789
```

```
# start an Instance
```

```
user@aws:$ aws ec2 stop-instances --instance-ids i-0123456789
```

```
# stop an Instance
```

```
user@aws:$ aws ec2 terminate-instances --instance-ids i-0123456789
```

```
# terminate an Instance
```

```
user@aws:$ aws ec2 describe-instance-status --include-all-instances
```

```
# check the health and reachability of instances
```

```
user@aws:$ aws ec2 get-console-output --instance-id i-0123456789
```

```
# retrieve the system logs, which can help diagnose boot issues.
```



S3

```
user@aws:$ aws s3 ls
```

```
# list all buckets
```

```
user@aws:$ aws s3 mb s3://my-new-bucket
```

```
# create a new bucket
```

```
user@aws:$ aws s3 rb s3://my-new-bucket --force
```

```
# delete a bucket
```

```
user@aws:$ aws s3 cp myfile.txt s3://my-bucket/
```

```
# copy a file to S3
```

```
user@aws:$ aws s3 cp s3://my-bucket/myfile.txt ./
```

```
# copy a file from S3 to local
```

```
user@aws:$ aws s3 cp s3://my-bucket/myfile.txt ./
```

```
# sync a local directory to S3
```

```
user@aws:$ aws s3 ls s3://my-bucket # list objects in a bucket
```



IAM

```
user@aws:$ aws iam list-users # list users
user@aws:$ aws iam create-user --user-name new-user
user@aws:$ aws iam list-roles # list roles
user@aws:$ aws iam attach-user-policy --user-name new-user --policy-
arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
# attach a policy to a user
user@aws:$ aws iam create-group --group-name MyUserGroup
# create a user group
user@aws:$ aws iam attach-group-policy --group-name MyUserGroup --
policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
# attach a policy to the group
user@aws:$ aws iam add-user-to-group --user-name MyUser --group-
name MyUserGroup # add a user to the group
```



CloudWatch

```
user@aws:$ aws cloudwatch describe-alarms
# list CloudWatch alarms
user@aws:$ aws cloudwatch put-metric-alarm --alarm-name MyAlarm --
metric-name CPUUtilization --namespace AWS/EC2 --statistic Average --
period 300 --threshold 80 --comparison-operator GreaterThanThreshold
--dimensions Name=InstanceId,Value=i-0123456789 --evaluation-
periods 2 --alarm-actions arn:aws:sns:eu-central-1:123456789012:my-
sns-topic
# create a CloudWatch alarm
user@aws:$ aws cloudwatch get-metric-data --metric-data-queries
file://queries.json --start-time 2023-01-01T00:00:00Z --end-time 2023-
01-02T00:00:00Z
# get CloudWatch metrics data
```



DynamoDB

```
user@aws:$ aws dynamodb list-tables
# list DynamoDB tables
user@aws:$ aws dynamodb describe-table --table-name MyTable
# describe a DynamoDB table
user@aws:$ aws dynamodb put-item --table-name MyTable --item '{"Id":
{"S": "123"}, "Name": {"S": "AWS"}}'
# put an item into a table
user@aws:$ aws dynamodb query --table-name MyTable --key-
condition-expression "Id = :id" --expression-attribute-values '{":id": {"S":
"123"}}'
# query a table
```



ECS

```
user@aws:$ aws ecs list-clusters                # list ECS clusters
user@aws:$ aws ecs list-tasks --cluster MyCluster
# list tasks in a cluster
user@aws:$ aws ecs describe-services --cluster MyCluster --services
MyService
# describe ECS services in a cluster
user@aws:$ aws ecs create-cluster --cluster-name MyCluster
# create an ECS cluster
user@aws:$ aws ecs create-service --cluster MyCluster --service-name
my-service --task-definition my-task --desired-count 2 --launch-type
FARGATE --network-configuration "awsvpcConfiguration={subnets=
[subnet-12345],securityGroups=[sg-12345],assignPublicIp=ENABLED}"
# if task defined, create an ECS service
```



ECR

```
user@aws:$ aws ecr describe-repositories
# list of all repositories
user@aws:$ aws ecr list-images --repository-name my-repo
# view all images in a repository
user@aws:$ aws ecr describe-images --repository-name my-repo --
image-ids imageTag=latest
# get details about a specific image, including the image size, push date
user@aws:$ aws ecr create-repository --repository-name my-repo
# create an ECR Repository
user@aws:$ aws ecr get-login-password --region eu-central-1 | docker
login --username AWS --password-stdin <aws_account_id>.dkr.ecr.us-
west-2.amazonaws.com
# get ECR login password
```



Lambda

```
user@aws:$ aws lambda create-function --function-name MyFunction --  
runtime python3.12 --role arn:aws:iam::123456789012:role/MyRole --  
handler my_function.handler --zip-file fileb://function.zip  
# create a Lambda function  
user@aws:$ aws lambda update-function-code --function-name  
MyFunction --zip-file fileb://new-function.zip  
# update a Lambda function code  
user@aws:$ aws lambda list-functions          # list Lambda functions  
user@aws:$ aws logs tail /aws/lambda/my-function --follow  
# retrieve recent logs for a Lambda function  
user@aws:$ aws lambda invoke --function-name my-function  
output.json  
# run a test event to see if the function executes correctly
```



SNS

```
user@aws:$ aws sns list-topics # list SNS topics
user@aws:$ aws sns list-subscriptions-by-topic --topic-arn
arn:aws:sns:eu-central-1:123456789012:my-topic
# view all subscribers of a specific topic
user@aws:$ aws sns create-topic --name my-topic
# create an SNS topic
user@aws:$ aws sns subscribe --topic-arn arn:aws:sns:eu-central-
1:123456789012:my-topic --protocol email --notification-endpoint
myemail@example.com
# subscribe to an SNS topic
user@aws:$ aws sns publish --topic-arn arn:aws:sns:us-west-
2:123456789012:my-topic --message "Test message"
# send a test message to an SNS topic
```



SQS

```
user@aws:$ aws sqs list-queues
```

```
# list queues
```

```
user@aws:$ aws sqs create-queue --queue-name my-queue
```

```
# create a queue
```

```
user@aws:$ aws sqs send-message --queue-url https://sqs.eu-central-1.amazonaws.com/123456789012/my-queue --message-body "Hello World"
```

```
# send a message to a queue
```

```
user@aws:$ aws sqs receive-message --queue-url https://sqs.us-west-2.amazonaws.com/123456789012/my-queue
```

```
# receive messages from a queue
```



Secrets Manager

```
user@aws:$ aws secretsmanager list-secrets
# get a list of all secrets to confirm existence
user@aws:$ aws secretsmanager get-secret-value --secret-id my-secret
# access the actual value of a secret to troubleshoot access issues
user@aws:$ aws secretsmanager create-secret --name my-secret --
secret-string '{"username":"admin","password":"password"}'
# create a Secret
```



CloudTrail

```
user@aws:$ aws cloudtrail describe-trails
```

```
# list all CloudTrail trails
```

```
user@aws:$ aws cloudtrail start-logging --name my-trail
```

```
# start logging on a Trail
```

```
user@aws:$ aws cloudtrail stop-logging --name my-trail
```

```
# stop logging on a Trail
```



VPC

```
user@aws:$ aws ec2 describe-vpcs
```

```
# list VPCs
```

```
user@aws:$ aws ec2 describe-subnets
```

```
# get details of subnets, including available IP addresses and AZs
```

```
user@aws:$ aws ec2 describe-network-interfaces
```

```
# view all ENIs, including their status and attachments
```



EBS

```
user@aws:$ aws ec2 describe-volumes --query "Volumes[*].  
[VolumeId,Size,State,AvailabilityZone]"  
# get a list of all EBS volumes, including their sizes, states, and AZ  
user@aws:$ aws ec2 describe-volume-status  
# describe volume status  
user@aws:$ aws ec2 describe-snapshots --owner-ids self  
# get details of EBS snapshots for backup verification
```



CloudFront

```
user@aws:$ aws cloudfront list-distributions
# view all CloudFront distributions, including their status, domain names
user@aws:$ aws cloudfront get-distribution --id E1234567890
# check configuration details of a specific CloudFront distribution
user@aws:$ aws cloudfront create-invalidation --distribution-id
E1234567890 --paths "/*"
# clear cached objects in a CloudFront distribution to troubleshoot
# outdated content.
```



ELB

```
user@aws:$ aws elb describe-load-balancers
# view all load balancers and their configurations to verify status
user@aws:$ aws elb describe-instance-health --load-balancer-name my-
load-balancer
# check the health status of registered instances in a load balancer
user@aws:$ aws elbv2 describe-target-groups
# list Target Groups (for ALB/NLB)
```



CloudFormation

```
user@aws:$ aws cloudformation describe-stacks
# check all CloudFormation stacks and their status
user@aws:$ aws cloudformation describe-stack-events --stack-name
MyStack
# recent events for a stack, useful for identifying stack creation issues
user@aws:$ aws cloudformation validate-template --template-body
file://template.json
# validate the syntax of CloudFormation template before deploying it
```



RDS

```
user@aws:$ aws rds describe-db-instances
# view all RDS instances and their status
user@aws:$ aws rds download-db-log-file-portion --db-instance-
identifier my-db-instance --log-file-name error/mysql-error.log --
starting-token 0
# retrieve logs for an RDS instance to help troubleshoot issues
```



```
user@aws:$ #####
```

Follow for Tips on AWS, K8s, Docker, Linux
Terraform, Ansible, DevOps

Why? Cause; More will unfold over time

||

∨

<https://linkedin.com/in/omerberatsezer>

<https://github.com/omerbsezer>

Feel free to like, share, or repost
to help more people see it

```
user@aws:$ #####
```



linkedin.com/in/omerberatsezer